Hummingbot AMM Arbitrage 策略深度解析

作者: AI Assistant

日期: 2025-10-28

策略版本: V1 (Legacy Strategy)

📚 目录

- 策略概述
- 核心架构
  - 类结构与继承关系
  - 关键属性
  - 依赖组件
- 策略生命周期
  - 初始化阶段
  - 就绪检查
  - 主循环执行
- 套利执行流程
  - 套利提案生成
  - 盈利性筛选
  - 滑点缓冲应用
  - 预算约束检查
  - 订单执行
- 数学模型
  - 利润计算
  - 滑点建模
  - 费用结构
- 配置参数详解
- 风险管理
  - 执行风险
  - 余额风险
  - 网络风险
- 执行模式对比
  - 并发执行模式

    - 顺序执行模式
- DEX 特殊处理
    - Gas 费用计算
    - EVM 优先级
- 实战建议
- 常见问题
- 附录：代码片段解析

策略概述

AMM Arbitrage 是 Hummingbot 的核心套利策略之一，适用于中心化交易所（CEX）、去中心化交易所（DEX）和自动做市商（AMM）之间的价格差异套利。

策略特点

特性	说明
通用性	支持 CEX-CEX、CEX-DEX、DEX-DEX 所有组合
实时性	每秒检查套利机会（tick = 1 秒）
双向扫描	同时评估买入/卖出两个方向
风险控制	内置滑点保护、余额检查、网络监控
执行模式	支持并发/顺序两种执行方式

核心逻辑

```
Market 1（买）←→ Market 2（卖）
或
Market 1（卖）←→ Market 2（买）
```

当满足以下条件时执行套利：

$$\text{净利润率} = \frac{P_{\text{sell}} \times (1 - f_{\text{sell}}) - P_{\text{buy}} \times (1 + f_{\text{buy}}) - \text{Gas}}{P_{\text{buy}} \times (1 + f_{\text{buy}})} \geq \text{min_profitability}$$

核心架构

类结构与继承关系

```
StrategyPyBase（基类）
    ↓
AmmArbStrategy（套利策略）
```

继承的核心功能：

- 订单跟踪（`_sb_order_tracker`）
- 市场管理（`add_markets`）
- 余额查询（`wallet_balance_data_frame`）
- 订单执行（`buy_with_specific_market` / `sell_with_specific_market`）

关键属性

```
# 市场配置
_market_info_1: MarketTradingPairTuple  # 市场 1 信息
_market_info_2: MarketTradingPairTuple  # 市场 2 信息

# 策略参数
_min_profitability: Decimal              # 最低盈利阈值
_order_amount: Decimal                   # 订单数量
_market_1_slippage_buffer: Decimal       # 市场 1 滑点缓冲
_market_2_slippage_buffer: Decimal       # 市场 2 滑点缓冲
_concurrent_orders_submission: bool      # 是否并发提交订单

# 运行时状态
_all_arb_proposals: Optional[List[ArbProposal]]  # 当前所有套利提案
_all_markets_ready: bool                         # 市场是否就绪
_main_task: Optional[asyncio.Task]               # 主任务
_order_id_side_map: Dict[str, ArbProposalSide]   # 订单 ID 到交易方向的映射

# 汇率转换
_rate_source: Optional[RateOracle]       # 汇率源（RateOracle 或 FixedRateSource）
```

依赖组件

1. MarketTradingPairTuple

```
(market, trading_pair, base_asset, quote_asset)
```

封装了市场连接器和交易对信息。

2. ArbProposal

```
class ArbProposal:
    first_side: ArbProposalSide   # 第一笔交易
    second_side: ArbProposalSide  # 第二笔交易

    def profit_pct(rate_source, account_for_fee) -> Decimal:
        # 计算利润百分比
```

3. ArbProposalSide

```
class ArbProposalSide:
    market_info: MarketTradingPairTuple
    is_buy: bool
    order_price: Decimal
    amount: Decimal
    completed_event: asyncio.Event
    is_failed: bool
```

4. RateOracle

用于不同报价资产间的汇率转换（如 WETH ↔ USDT）。

策略生命周期

初始化阶段

1. `init_params()` 方法（第 69-119 行）

```
def init_params(self,
                market_info_1: MarketTradingPairTuple,
                market_info_2: MarketTradingPairTuple,
                min_profitability: Decimal,
                order_amount: Decimal,
                market_1_slippage_buffer: Decimal = Decimal("0"),
                market_2_slippage_buffer: Decimal = Decimal("0"),
                concurrent_orders_submission: bool = True,
                status_report_interval: float = 900,
                rate_source: Optional[RateOracle] = RateOracle.get_instance()):
```

关键初始化步骤：

1. **保存策略参数**

   ```
   self._market_info_1 = market_info_1
   self._market_info_2 = market_info_2
   self._min_profitability = min_profitability
   self._order_amount = order_amount
   ```

2. **初始化运行时状态**

   ```
   self._all_arb_proposals = None
   self._all_markets_ready = False
   self._main_task = None
   self._order_id_side_map: Dict[str, ArbProposalSide] = {}
   ```

3. **注册市场**

   ```
   self.add_markets([market_info_1.market, market_info_2.market])
   ```

4. **设置汇率源**

   ```
   self._rate_source = rate_source
   ```

就绪检查

`tick()` 方法（第 173-192 行）

核心逻辑：

```
def tick(self, timestamp: float):
    # 1. 检查所有市场是否就绪
    if not self.all_markets_ready:
        self.all_markets_ready = all([market.ready for market in self.active_markets])
        if not self.all_markets_ready:
            # 每 10 秒报告未就绪的市场
            if int(timestamp) % 10 == 0:
                unready_markets = [market for market in self.active_markets
                                   if market.ready is False]
                for market in unready_markets:
                    msg = ', '.join([k for k, v in market.status_dict.items()
                                     if v is False])
                    self.logger().warning(f"{market.name} not ready: waiting for {msg}.
            return
        else:
            self.logger().info("Markets are ready. Trading started.")

    # 2. 检查是否可以开始新的套利交易
    if self.ready_for_new_arb_trades():
        if self._main_task is None or self._main_task.done():
            self._main_task = safe_ensure_future(self.main())
```

市场就绪条件（`market.ready`）：

- 订单簿已加载
- 余额已同步
- 网络连接正常
- （DEX）合约地址已解析

主循环执行

`ready_for_new_arb_trades()` 方法（第 342-350 行）

```python
def ready_for_new_arb_trades(self) -> bool:
    """
    返回 True 如果没有未完成的订单
    """
    for market_info in [self._market_info_1, self._market_info_2]:
        if len(self.market_info_to_active_orders.get(market_info, [])) > 0:
            return False
    return True
```

保护机制：

- 只有当两个市场都没有活跃订单时，才允许开始新的套利
- 避免订单冲突和资金占用

套利执行流程

流程图

```
┌──────────────────────────┐
│  tick() 每秒调用          │
└──────────────────────────┘
            │
            ↓
┌──────────────────────────┐
│  市场就绪检查              │
└──────────────────────────┘
            │
            ↓
┌──────────────────────────┐
│  无活跃订单检查            │
└──────────────────────────┘
            │
            ↓
┌──────────────────────────┐
│  main() 主流程            │
└──────────────────────────┘
            │
            ↓
┌──────────────────────────────────┐
│  1. create_arb_proposals()        │
│     - 获取市场报价                  │
│     - 生成买/卖双向提案              │
│     - 计算 Gas 费用（DEX）          │
└──────────────────────────────────┘
            │
            ↓
┌──────────────────────────────────┐
│  2. 筛选盈利提案                    │
│     profit_pct >= min_profitability │
└──────────────────────────────────┘
            │
            ↓
┌──────────────────────────────────┐
│  3. apply_slippage_buffers()      │
│     - 调整订单价格                  │
│     - 买单价格 × (1 + buffer)       │
│     - 卖单价格 × (1 - buffer)       │
└──────────────────────────────────┘
            │
            ↓
┌──────────────────────────────────┐
│  4. apply_budget_constraint()     │
│     - 检查余额是否充足              │
```

```
│      - 不足则将 amount 设为 0          │
└──────────────────────────────────────┘
              │
              ↓
┌──────────────────────────────────────┐
│  5. execute_arb_proposals()          │
│      - 并发模式：同时提交两个订单       │
│      - 顺序模式：等第一单成交后提交     │
└──────────────────────────────────────┘
```

套利提案生成

`create_arb_proposals()` 函数（第 200-214 行）

```
self._all_arb_proposals = await create_arb_proposals(
    market_info_1=self._market_info_1,
    market_info_2=self._market_info_2,
    market_1_extra_flat_fees=(
        [getattr(self._market_info_1.market, "network_transaction_fee")]
        if hasattr(self._market_info_1.market, "network_transaction_fee")
        else []
    ),
    market_2_extra_flat_fees=(
        [getattr(self._market_info_2.market, "network_transaction_fee")]
        if hasattr(self._market_info_2.market, "network_transaction_fee")
        else []
    ),
    order_amount=self._order_amount,
)
```

该函数做了什么（位于 `utils.py`）：

1. **获取市场报价**

   ```
   quote_1_buy = await market_1.get_quote_price(trading_pair_1, True, order_amount)
   quote_1_sell = await market_1.get_quote_price(trading_pair_1, False, order_amount)
   quote_2_buy = await market_2.get_quote_price(trading_pair_2, True, order_amount)
   quote_2_sell = await market_2.get_quote_price(trading_pair_2, False, order_amount)
   ```

2. **生成两个方向的提案**

   方向 1：市场 1 买入，市场 2 卖出

```
proposal_1 = ArbProposal(
    first_side=ArbProposalSide(
        market_info=market_info_1,
        is_buy=True,
        order_price=quote_1_buy,
        amount=order_amount
    ),
    second_side=ArbProposalSide(
        market_info=market_info_2,
        is_buy=False,
        order_price=quote_2_sell,
        amount=order_amount
    )
)
```

方向 2：市场 1 卖出，市场 2 买入

```
proposal_2 = ArbProposal(
    first_side=ArbProposalSide(
        market_info=market_info_1,
        is_buy=False,
        order_price=quote_1_sell,
        amount=order_amount
    ),
    second_side=ArbProposalSide(
        market_info=market_info_2,
        is_buy=True,
        order_price=quote_2_buy,
        amount=order_amount
    )
)
```

3. **计算每个提案的利润**
   包含：
   - 价格差异
   - 交易手续费
   - Gas 费用（DEX）
   - 汇率转换（不同报价资产）

盈利性筛选

第 215-227 行

```
profitable_arb_proposals: List[ArbProposal] = [
    t.copy() for t in self._all_arb_proposals
    if t.profit_pct(
        rate_source=self._rate_source,
        account_for_fee=True,
    ) >= self._min_profitability
]

if len(profitable_arb_proposals) == 0:
    if self._last_no_arb_reported < self.current_timestamp - 20.:
        self.logger().info("No arbitrage opportunity.\n" +
                           "\n".join(self.short_proposal_msg(self._all_arb_proposals, F
        self._last_no_arb_reported = self.current_timestamp
    return
```

关键点：

- 只保留利润率 ≥ `min_profitability` 的提案
- 无机会时，每 20 秒报告一次（避免日志刷屏）
- 报告包含两个方向的利润率

滑点缓冲应用

`apply_slippage_buffers()` 方法（第 232-249 行）

```
async def apply_slippage_buffers(self, arb_proposals: List[ArbProposal]):
    for arb_proposal in arb_proposals:
        for arb_side in (arb_proposal.first_side, arb_proposal.second_side):
            market = arb_side.market_info.market

            # 1. 量化订单数量（符合交易所精度要求）
            arb_side.amount = market.quantize_order_amount(
                arb_side.market_info.trading_pair,
                arb_side.amount
            )

            # 2. 确定滑点缓冲
            s_buffer = (self._market_1_slippage_buffer
                        if market == self._market_info_1.market
                        else self._market_2_slippage_buffer)

            # 3. 应用滑点（买单加价，卖单降价）
            if not arb_side.is_buy:
                s_buffer *= Decimal("-1")
            arb_side.order_price *= Decimal("1") + s_buffer

            # 4. 量化订单价格
            arb_side.order_price = market.quantize_order_price(
                arb_side.market_info.trading_pair,
                arb_side.order_price
            )
```

滑点缓冲的作用：

订单类型	原价格	滑点 1%	调整后价格	目的
买单 (BUY)	100 USDT	+1%	101 USDT	提高成交率
卖单 (SELL)	100 USDT	-1%	99 USDT	提高成交率

为什么需要滑点缓冲?

- CEX：订单簿价格可能快速变化
- DEX：交易需要等待区块确认（5-30 秒），期间价格可能滑动

- AMM：大额订单会导致价格冲击

预算约束检查

`apply_budget_constraint()` 方法（第 251-268 行）

```
def apply_budget_constraint(self, arb_proposals: List[ArbProposal]):
    for arb_proposal in arb_proposals:
        for arb_side in (arb_proposal.first_side, arb_proposal.second_side):
            market = arb_side.market_info.market

            # 1. 确定需要的代币
            token = (arb_side.market_info.quote_asset if arb_side.is_buy
                    else arb_side.market_info.base_asset)

            # 2. 获取可用余额
            balance = market.get_available_balance(token)

            # 3. 计算所需余额
            required = (arb_side.amount * arb_side.order_price if arb_side.is_buy
                       else arb_side.amount)

            # 4. 余额不足则放弃该提案
            if balance < required:
                arb_side.amount = s_decimal_zero
                self.logger().info(
                    f"Can't arbitrage, {market.display_name} {token} balance "
                    f"({balance}) is below required order amount ({required})."
                )
                continue
```

余额检查示例：

假设交易对为 ETH-USDT，订单量为 1 ETH：

订单类型	需要检查的余额	所需数量
买入 ETH	USDT 余额	1 × 价格 = 3000 USDT
卖出 ETH	ETH 余额	1 ETH

订单执行

`execute_arb_proposals()` 方法（第 289-329 行）

```
async def execute_arb_proposals(self, arb_proposals: List[ArbProposal]):
    for arb_proposal in arb_proposals:
        # 1. 跳过无效提案 (amount = 0 的)
        if any(p.amount <= s_decimal_zero
               for p in (arb_proposal.first_side, arb_proposal.second_side)):
            continue

        # 2. 顺序模式下，优先执行 EVM 链交易
        if not self._concurrent_orders_submission:
            arb_proposal = self.prioritize_evm_exchanges(arb_proposal)

        self.logger().info(f"Found arbitrage opportunity!: {arb_proposal}")

        # 3. 执行两笔订单
        for arb_side in (arb_proposal.first_side, arb_proposal.second_side):
            side: str = "BUY" if arb_side.is_buy else "SELL"
            self.log_with_clock(logging.INFO,
                                f"Placing {side} order for {arb_side.amount} "
                                f"{arb_side.market_info.base_asset} "
                                f"at {arb_side.market_info.market.display_name} "
                                f"at {arb_side.order_price} price")

            order_id: str = await self.place_arb_order(
                arb_side.market_info,
                arb_side.is_buy,
                arb_side.amount,
                arb_side.order_price
            )

            # 4. 保存订单 ID 映射
            self._order_id_side_map.update({order_id: arb_side})

            # 5. 顺序模式：等待第一个订单完成
            if not self._concurrent_orders_submission:
                await arb_side.completed_event.wait()
                if arb_side.is_failed:
                    self.log_with_clock(logging.ERROR,
                                        f"Order {order_id} failed. "
                                        f"Dropping Arbitrage Proposal.")
```

```
            return

    # 6. 等待套利完成
    await arb_proposal.wait()
```

数学模型

利润计算

基本公式

$$\Pi_{\text{raw}} = Q \times (P_{\text{sell}} - P_{\text{buy}})$$

其中：

- Q = 订单数量（`order_amount`）
- P_{sell} = 卖出价格
- P_{buy} = 买入价格

考虑费用后的净利润

$$\Pi_{\text{net}} = Q \times P_{\text{sell}} \times (1 - f_{\text{sell}}) - Q \times P_{\text{buy}} \times (1 + f_{\text{buy}}) - G$$

其中：

- f_{sell} = 卖出手续费率
- f_{buy} = 买入手续费率
- G = Gas 费用（DEX）

利润率

$$\text{Profit} = \frac{\Pi_{\text{net}}}{Q \times P_{\text{buy}} \times (1 + f_{\text{buy}})} \times 100\%$$

盈利条件：

$$\text{Profit} \geq \text{min_profitability}$$

滑点建模

应用滑点后的价格

买单：

$$P_{\text{buy}}^{\text{adj}} = P_{\text{buy}} \times (1 + s_{\text{buffer}})$$

卖单：

$$P_{\text{sell}}^{\text{adj}} = P_{\text{sell}} \times (1 - s_{\text{buffer}})$$

滑点后的净利润

$$\Pi_{\text{net}}^{\text{slip}} = Q \times P_{\text{sell}}^{\text{adj}} \times (1 - f_{\text{sell}}) - Q \times P_{\text{buy}}^{\text{adj}} \times (1 + f_{\text{buy}}) - G$$

费用结构

CEX 费用

交易所	Maker 费率	Taker 费率	提款费用
Binance	0.10%	0.10%	动态
Gate.io	0.15%	0.15%	动态
MEXC	0.00%	0.20%	动态

DEX/AMM 费用

协议	交易费率	Gas 费用	价格冲击
Uniswap V2	0.30%	~150k gas	高
Uniswap V3	0.05%/0.3%/1%	~180k gas	中
PancakeSwap	0.25%	~120k gas	中

Gas 费用计算（DEX）

```
# 获取网络 Gas 费用
network_transaction_fee = market.network_transaction_fee
# TokenAmount(token='ETH', amount=Decimal('0.0001'))

# 转换为报价资产
gas_in_quote = gas_amount × eth_price
```

示例：

- Gas 使用：180,000 gas
- Gas 价格：30 Gwei
- ETH 价格：$3,800
- Gas 费用：0.0054 ETH = $20.52

配置参数详解

必需参数

1. `market_info_1` / `market_info_2`

- **类型**： `MarketTradingPairTuple`
- **说明**：两个市场的配置
- **示例**：

  ```
  market_info_1 = (uniswap_market, "IRON-WETH", "IRON", "WETH")
  market_info_2 = (gateio_market, "IRON-USDT", "IRON", "USDT")
  ```

2. `min_profitability`

- **类型**： `Decimal`
- **范围**：0.001 - 0.10（0.1% - 10%）
- **推荐值**：
  - CEX-CEX：0.003 - 0.005（0.3% - 0.5%）
  - CEX-DEX：0.01 - 0.02（1% - 2%）
  - DEX-DEX：0.015 - 0.03（1.5% - 3%）
- **说明**：最低盈利阈值，用于筛选套利机会

3. `order_amount`

- **类型**： `Decimal`
- **单位**：基础资产数量
- **说明**：每次套利的订单数量
- **建议**：根据池子流动性设置（参考前面 PING 的例子）

可选参数

4. `market_1_slippage_buffer`

- **类型**： `Decimal`
- **默认值**： `0`
- **范围**：0 - 0.05（0% - 5%）
- **推荐值**：
  - CEX：0.001 - 0.003（0.1% - 0.3%）
  - DEX/AMM：0.01 - 0.03（1% - 3%）
- **说明**：市场 1 的滑点保护缓冲

5. `market_2_slippage_buffer`

- **类型**： `Decimal`
- **默认值**： `0`
- **说明**：市场 2 的滑点保护缓冲

6. `concurrent_orders_submission`

- **类型**： `bool`
- **默认值**： `True`
- **说明**：
  - `True` ： 并发执行（同时提交两个订单）
  - `False` ： 顺序执行（等第一单成交后再提交第二单）
- **推荐**：
  - CEX-CEX： `True`
  - CEX-DEX： `False` （DEX 确认慢）
  - DEX-DEX： `False`

7. `status_report_interval`

- **类型**： `float`

- **默认值**： `900` （秒）
- **说明**：状态报告刷新间隔

8. `rate_source`

- **类型**： `RateOracle`
- **默认值**： `RateOracle.get_instance()`
- **说明**：汇率源，用于不同报价资产间转换

风险管理

执行风险

1. 部分成交风险

风险：只有一边订单成交，另一边失败

保护机制（第 321-327 行）：

```
if not self._concurrent_orders_submission:
    await arb_side.completed_event.wait()
    if arb_side.is_failed:
        self.log_with_clock(logging.ERROR,
                            f"Order {order_id} failed. Dropping Arbitrage Proposal.")
        return  # 放弃第二笔订单
```

顺序执行模式的优势：

- ✅ 第一单失败时，不会提交第二单
- ✅ 避免单边持仓风险
- ❌ 执行速度慢，可能错过机会

2. 价格滑动风险

风险：提交订单后，价格变化导致利润消失甚至亏损

保护机制：

- 滑点缓冲（ `slippage_buffer` ）

- 市场订单（Taker 订单，立即成交）

3. Gas 费用波动风险（DEX）

风险：网络拥堵时 Gas 价格飙升

保护机制：

- 实时获取 `network_transaction_fee`
- 利润计算中考虑 Gas 成本

余额风险

`apply_budget_constraint()` 保护

检查时机：每次生成套利提案后

检查内容：

1. 买单：检查报价资产余额
2. 卖单：检查基础资产余额

失败处理：

- 将 `amount` 设为 0
- 跳过该提案
- 记录警告日志

网络风险

`network_warning()` 方法

定期检查：

- 网络连接状态
- 订单簿更新时间
- API 响应延迟

执行模式对比

并发执行模式

配置

```yaml
concurrent_orders_submission: true
```

执行流程

```
T0: 提交订单 1（Market 1 买入）
T0: 提交订单 2（Market 2 卖出）
    ↓
T1: 等待两个订单都成交
```

优势

- ✅ 执行速度快
- ✅ 捕捉短暂套利机会
- ✅ 减少价格滑动风险

劣势

- ❌ 可能出现单边成交
- ❌ 需要更多初始资金

适用场景

- CEX-CEX 套利
- 高流动性市场
- 价格差异波动快

顺序执行模式

配置

```yaml
concurrent_orders_submission: false
```

执行流程

```
T0: 提交订单 1（优先 DEX）
    ↓
T1: 等待订单 1 成交
    ↓
T2: 检查订单 1 状态
    ↓
T3: 提交订单 2（如果订单 1 成功）
    ↓
T4: 等待订单 2 成交
```

优势

- ✅ 避免单边持仓风险
- ✅ 资金利用率高
- ✅ 适合慢速网络

劣势

- ❌ 执行慢，可能错过机会
- ❌ 价格可能在等待期间变化

适用场景

- CEX-DEX 套利
- DEX-DEX 套利
- 低流动性市场

DEX 特殊处理

Gas 费用计算

代码实现（第 203-211 行）

```
market_1_extra_flat_fees=(
    [getattr(self._market_info_1.market, "network_transaction_fee")]
    if hasattr(self._market_info_1.market, "network_transaction_fee")
    else []
),
market_2_extra_flat_fees=(
    [getattr(self._market_info_2.market, "network_transaction_fee")]
    if hasattr(self._market_info_2.market, "network_transaction_fee")
    else []
),
```

Gas 费用结构

```
TokenAmount(
    token='ETH',            # Gas 代币
    amount=Decimal('0.0054')  # Gas 数量
)
```

转换为报价资产

在 `ArbProposal.profit_pct()` 中：

```
# 假设 Gas 是 ETH，报价资产是 USDT
gas_in_eth = Decimal('0.0054')
eth_to_usdt_rate = rate_source.get_pair_rate('ETH-USDT')  # 3800
gas_in_usdt = gas_in_eth × eth_to_usdt_rate  # 20.52 USDT
```

EVM 优先级

`prioritize_evm_exchanges()` 方法（第 270-287 行）

```python
def prioritize_evm_exchanges(self, arb_proposal: ArbProposal) -> ArbProposal:
    results = []
    for side in [arb_proposal.first_side, arb_proposal.second_side]:
        if self.is_gateway_market(side.market_info):
            results.insert(0, side)  # EVM 交易放在前面
        else:
            results.append(side)      # CEX 交易放在后面

    return ArbProposal(first_side=results[0], second_side=results[1])
```

为什么优先执行 DEX 订单?

1. **交易确认时间长**
   - DEX：5-30 秒（等待区块确认）
   - CEX：<1 秒（链下撮合）
2. **降低价格风险**
   - 先锁定慢速交易
   - CEX 可以快速反应价格变化
3. **示例**

```
顺序 1（推荐）: DEX 买入 → CEX 卖出
- T0: 提交 DEX 买单
- T15: DEX 成交（15 秒后）
- T16: 提交 CEX 卖单
- T17: CEX 成交（1 秒后）

顺序 2（不推荐）: CEX 买入 → DEX 卖出
- T0: 提交 CEX 买单
- T1: CEX 成交（1 秒后）
- T2: 提交 DEX 卖单
- T17: DEX 成交（15 秒后）
- 风险：持有 15 秒现货，价格可能下跌
```

实战建议

1. 交易对选择

高流动性交易对（推荐）

- ETH-USDT / ETH-USDC
- BTC-USDT / WBTC-USDC
- 主流稳定币对

低流动性交易对（谨慎）

- 小市值代币
- 新上市代币
- DEX 独家代币

流动性评估指标:

```
# 检查订单簿深度
depth_1 = market_1.get_order_book_depth(trading_pair, 0.01)
depth_2 = market_2.get_order_book_depth(trading_pair, 0.01)

# 至少需要
min_depth = order_amount × 10  # 10 倍订单量
```

2. 参数调优

起始参数（保守）

```
order_amount: 10-50          # 小额测试
min_profitability: 0.02      # 2% 利润阈值
market_1_slippage_buffer: 0.02  # 2% 滑点保护
market_2_slippage_buffer: 0.02
concurrent_orders_submission: false  # 顺序执行
```

优化参数（激进）

```
order_amount: 500-1000        # 提高至流动性允许
min_profitability: 0.005      # 0.5% 利润阈值
market_1_slippage_buffer: 0.005  # 0.5% 滑点保护
market_2_slippage_buffer: 0.005
concurrent_orders_submission: true  # 并发执行
```

3. 监控指标

核心指标

```
# 1. 成功率
success_rate = successful_arbs / total_arbs

# 2. 平均利润率
avg_profit = sum(profits) / len(profits)

# 3. 执行时间
avg_exec_time = sum(exec_times) / len(exec_times)

# 4. Gas 费用占比（DEX）
gas_ratio = total_gas_cost / total_profit
```

警报阈值

- 成功率 < 80%：检查网络或余额
- 平均利润 < min_profitability × 1.5：提高阈值
- Gas 费用 > 利润的 50%：暂停 DEX 套利
- 执行时间 > 30 秒：检查网络延迟

4. 风险控制

单次套利风险限额

```
max_loss_per_arb = total_capital × 0.005  # 0.5% 总资金
max_order_amount = max_loss_per_arb / max_price_impact
```

总体风险限额

```
daily_loss_limit = total_capital × 0.05   # 5% 总资金
if daily_loss > daily_loss_limit:
    stop_trading()
```

常见问题

Q1: 为什么显示有套利机会但不执行?

可能原因:

1. **余额不足** - 检查 `apply_budget_constraint()` 日志
2. **滑点后利润消失** - 降低 `slippage_buffer`
3. **订单精度问题** - 检查 `quantize_order_amount()` 结果

Q2: DEX 交易总是失败?

可能原因:

1. **Gas 价格设置过低** - 提高 Gas 价格设置
2. **代币未授权** - 执行 `approve` 交易
3. **流动性不足** - 降低 `order_amount`
4. **滑点保护不足** - 提高 `slippage_buffer`

Q3: 利润率显示 10% 但实际亏损?

可能原因:

1. **未考虑 Gas 费用** - 检查 `network_transaction_fee`
2. **汇率转换错误** - 验证 `rate_source` 配置
3. **滑点缓冲过大** - 降低 `slippage_buffer`
4. **价格冲击严重** - 降低 `order_amount`

Q4: 如何提高套利成功率?

优化建议:

1. **使用顺序执行模式** - `concurrent_orders_submission: false`
2. **提高滑点保护** - 增加 `slippage_buffer`
3. **选择高流动性交易对** - 避免价格冲击
4. **降低利润阈值** - 增加机会数量（但降低每次利润）
5. **优化网络** - 使用 VPS 接近交易所位置

Q5: 适合什么网络环境运行?

推荐配置：

- **网络延迟**：< 50ms 到交易所
- **带宽**：至少 10 Mbps
- **稳定性**：99.9% 可用性
- **位置**：VPS 靠近主要交易所（如新加坡、东京）

附录：代码片段解析

A1: 订单完成事件处理

```
def did_complete_buy_order(self, order_completed_event: BuyOrderCompletedEvent):
    # 1. 标记订单完成
    self.set_order_completed(order_id=order_completed_event.order_id)

    # 2. 获取市场信息
    market_info: MarketTradingPairTuple = self.order_tracker.get_market_pair_from_order
        order_completed_event.order_id
    )

    # 3. 记录日志
    log_msg: str = f"Buy order completed on {market_info.market.name}: {order_completed
    if self.is_gateway_market(market_info):
        log_msg += f" txHash: {order_completed_event.exchange_order_id}"
    self.log_with_clock(logging.INFO, log_msg)

    # 4. 发送通知
    self.notify_hb_app_with_timestamp(
        f"Bought {order_completed_event.base_asset_amount:.8f} "
        f"{order_completed_event.base_asset}-{order_completed_event.quote_asset} "
        f"on {market_info.market.name}."
    )
```

A2: 订单 ID 到交易方向的映射

```
# 提交订单时保存映射
self._order_id_side_map.update({
    order_id: arb_side
})

# 订单完成时查找
arb_side: Optional[ArbProposalSide] = self._order_id_side_map.get(order_id)
if arb_side:
    arb_side.set_completed()  # 触发 completed_event
```

用途：

- 跟踪每个订单对应的套利提案
- 顺序执行模式中等待第一个订单完成
- 统计套利成功率

A3: 状态报告生成

```
async def format_status(self) -> str:
    # 1. 市场价格
    columns = ["Exchange", "Market", "Sell Price", "Buy Price", "Mid Price"]
    data = []
    for market_info in [self._market_info_1, self._market_info_2]:
        buy_price = await market.get_quote_price(trading_pair, True, self._order_amount
        sell_price = await market.get_quote_price(trading_pair, False, self._order_amou
        mid_price = (buy_price + sell_price) / 2
        data.append([market.display_name, trading_pair, sell_price, buy_price, mid_pric

    # 2. Gas 费用 (DEX)
    for market_info in [self._market_info_1, self._market_info_2]:
        if hasattr(market_info.market, "network_transaction_fee"):
            transaction_fee: TokenAmount = getattr(market_info.market, "network_transac
            data.append([market_info.market.display_name,
                         f"{transaction_fee.amount} {transaction_fee.token}"])

    # 3. 资产余额
    assets_df = self.wallet_balance_data_frame([self._market_info_1, self._market_info_

    # 4. 盈利性分析
    profitability = self.short_proposal_msg(self._all_arb_proposals)

    # 5. 汇率信息
    fixed_rates_df = self.get_fixed_rates_df()

    # 6. 警告信息
    warnings = self.network_warning([self._market_info_1, self._market_info_2])
    warnings.extend(self.balance_warning([self._market_info_1, self._market_info_2]))

    return formatted_status
```

总结

AMM Arbitrage 策略是一个**通用、稳健、高效**的套利策略，适用于几乎所有类型的交易所组合。通过理解其核心机制、风险控制和参数优化，你可以在各种市场条件下稳定盈利。

关键要点：

1. ✅ 从小额、保守参数开始测试
2. ✅ 选择高流动性交易对
3. ✅ DEX 套利使用顺序执行模式
4. ✅ 持续监控关键指标
5. ✅ 设置严格的风险限额

进阶学习：

- 研究 `create_arb_proposals()` 在 `utils.py` 中的实现
- 学习 `ArbProposal` 的利润计算逻辑
- 了解 `RateOracle` 的汇率转换机制
- 探索 Gateway 的 DEX 连接实现

祝你套利成功！🚀